

Simulating Multi-Stage Interconnection Networks in Python

Ferola

Sapienza University of Rome
Advanced Architectures

hima.2060001@studenti.uniroma1.it

Abstract

This project builds and tests four interconnection networks used in parallel computers: Baseline, Beneš, Clos, and XGFT. Each network is built in Python, given a routing method, and checked for correctness. The Clos network is also tested for blocking, and all four are compared by size and cost.

1 Task description

An interconnection network connects many processors (or switches) together inside a parallel computer. This project studies four kinds of such networks and shows, in code, how data moves through each one from an input to an output.

1.1 Real-world applications

These networks are used to connect processors, memory, and servers in supercomputers and data centers, where many devices need fast, reliable paths to each other at once.

2 Related work

The Clos network was introduced by (author?) [2] as a way to build large switching networks from small switches, with a proven condition for avoiding blocking. The Beneš network, described by (author?) [1], showed that the same idea could route any full permutation with no clashes. Fat-tree networks, including the generalized form (XGFT) built here, are a common choice in today's data center networks.

3 Tools and methods

The project is written in Python, using the `networkx` library to build and check each network as a graph, and `matplotlib` to draw pictures of them. No external dataset is needed: each network is generated directly by code, with a

few example sizes used throughout (for example, Baseline and Beneš with 8 inputs; Clos with 4 input switches, 3 inputs each, and 5 middle switches; XGFT with height 2).

4 Evaluation approach

In the literature, these networks are usually judged by: how many switches and wires they need for a given size, how they are routed, and whether they can get blocked (unable to find a free path) under load. This project follows the same three measures.

5 Experiments

For each network, every possible input-output path was checked against the actual wires built, to confirm the routing code is correct. For Beneš, this was done using 60 random full permutations. For Clos, correctness was also checked for the non-blocking condition directly: with $m \geq 2n - 1$ middle switches, no random traffic pattern caused any blocked call, across many trials.

Switch and wire counts were also measured as network size grows, and, for Clos, as the number of middle switches changes.

5.1 Results

Beneš needs close to twice as many switches as Baseline for the same size, matching the theory (Beneš has $2 \log_2 N - 1$ stages, Baseline has $\log_2 N$). Both grow with $N \log N$, not N^2 , which is why these networks scale well.

For Clos, blocking rate drops sharply as the number of middle switches increases, reaching zero once $m \geq 2n - 1$, exactly as predicted by the non-blocking theorem. Below that point, blocking still happens sometimes, but less often than the worst-case guarantee would suggest, since random traffic is unlikely to hit the exact worst case.

5.2 Discussion

The routing methods used here are simple: Baseline and XGFT use direct rules based on the destination address, Clos always tries middle switches in a fixed order, and Beneš uses the Looping Algorithm only for full permutations. None of these adapt to traffic patterns, so a busier or more clever routing strategy could likely do better. Blocking was only measured for Clos; the other three networks were not tested this way.

6 Conclusions

This project built four common interconnection networks in Python, routed data through each one, and checked the results against known theory. All four networks and their routing methods worked correctly, and the results (switch counts, blocking rates) matched what the theory predicts. Future work could add blocking analysis for the other networks, smarter routing strategies, or connect these networks to an actual workload, such as an arithmetic unit, to see how the choice of network affects real performance.

References

- [1] Václav E. Beneš. *Mathematical Theory of Connecting Networks and Telephone Traffic*. Academic Press, 1965. [1](#)
- [2] Charles Clos. A study of non-blocking switching networks. *Bell System Technical Journal*, 32(2):406–424, 1953. [1](#)